

# Lab1: DataLab

复旦大学  
黄果 23301030064

## 一、实验步骤与结果

### 1. 实验环境与基本流程

实验在 Windows 的 WSL Ubuntu 环境中完成，使用 GCC 的 32 位编译支持。课程给出的 DataLab 评测工具包括 check\_ops.py、dlc、btest、ishow 和 fshow，其中 check\_ops.py 用于检查运算符、常数与操作数数量是否合法，btest 用于检查函数结果是否正确。

```
1. sudo apt-get update
2. sudo apt-get install -y gcc make gcc-multilib libc6-dev-i386 python3
```

每次修改 bits.c 后重新编译，避免 btest 仍测试旧的可执行文件：

```
1. make clean
2. make all
```

随后依次执行规则检查与正确性测试：

```
1. ./check_ops.py bits.c
2. ./btest
```

### 2. 测试结果

```
squaredog@DESKTOP-92P7DI3:~/Lab1$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 13/16 operators
roundEvenPow2: 11/24 operators
midpointTowardFirst: 21/32 operators
isBetweenEitherOrder: 45/48 operators
mul5Sat: 26/30 operators
classifyAdd3: 38/52 operators
floatScaleThreeHalves: 38/60 operators
floatRoundEven: 26/65 operators
float_i2f: 33/40 operators
bitCount: 28/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

图 1 执行 ./check\_ops.py bits.c 后的规则检查结果

```

squaredog@DESKTOP-92P7DI3:~/Lab1$ ./btest
Score  Rating  Errors  Function
1      1        0    signMask
2      2        0    bitXor
3      3        0    negativePart
4      4        0    copyByteWithin
4      4        0    logicalShift
4      4        0    swapNibblePairs
4      4        0    secondLowestZeroBit
5      5        0    oddParity
5      5        0    rotateRightBits
5      5        0    roundEvenPow2
5      5        0    midpointTowardFirst
7      7        0    isBetweenEitherOrder
7      7        0    mul5Sat
7      7        0    classifyAdd3
7      7        0    floatScaleThreeHalves
10     10        0    floatRoundEven
10     10        0    float_i2f
10     10        0    bitCount
10     10        0    bitReverse
Total points: 110/110

```

图 2 执行./btest 后的正确性测试结果

### 3. 各函数实现思路

整数题在 32 位补码、算术右移的模型下，只能使用题目允许的位运算与 0~255 范围内常数；浮点题允许使用 int/unsigned、循环和条件判断，但不能使用浮点类型、浮点运算、类型转换或额外函数。下面按 P1–P19 说明实现思路。

#### P1 signMask

目标是只保留最高有效位。利用 1 左移 31 位直接构造 0x80000000。

#### P2 bitXor

题目只允许使用~和&。根据异或“两个比特不同则为 1”的定义，先排除“同时为 1”，再排除“同时为 0”，两者相与后只保留不同的位，写成~(x&y)&~(~x&~y)。

#### P3 negativePart

利用算术右移得到符号掩码  $x \gg 31$ ：非负数得到全 0，负数得到全 1；对于一个 32 位整数， $x$  和  $\sim x$  的每一位正好相反，因此  $x + \sim x = 0xFFFFFFFF = -1$ ，即根据  $-x = \sim x + 1$  计算相反数，最后与符号掩码相与，从而使非负数返回 0、负数返回  $-x$ 。

#### P4 copyByteWithin

首先利用  $src \ll 3$  计算源字节的位偏移量，将  $x$  右移对应位数，使第  $src$  个字节移动到最低 8 位；再与掩码 0x000000FF 进行按位与，利用其最低 8 位为 1、其余位为 0 的特点保留源字节并清除其他高位。随后利用  $dst \ll 3$  计算目标字节偏移，并通过  $0xFF \ll dstShift$  构造

目标字节掩码，其可能为 0x000000FF、0x0000FF00、0x00FF0000 或 0xFF000000；对该掩码取反后与 x 相与，即可只清空目标字节而保留其他字节。最后将提取出的源字节左移 dstShift 位移动到目标位置，再与清零后的原数进行按位或运算，从而完成字节复制。

## P5 logicalShift

目标是在 C 语言默认算术右移的条件下实现逻辑右移。首先直接计算  $x \gg n$ ，此时若 x 为负数，左侧会因为符号扩展补入 n 个 1，因此需要额外构造一个高 n 位为 0、其余位为 1 的掩码。以最高位 1 << 31 为起点，先算术右移 n 位得到左侧连续的 1，再左移 1 位并按位取反，即可得到所需掩码。最后将  $x \gg n$  与该掩码相与，把算术右移补入的高位 1 全部清零，同时保留原本右移后的有效数据，从而得到逻辑右移结果。

## P6 swapNibblePairs

目标是交换每个字节内部的高 4 位和低 4 位。首先构造掩码 0x0F0F0F0F，其中每个字节的低 4 位为 1、高 4 位为 0，用它与 x 相与可以提取每个字节的低 4 位，再将这些位整体左移 4 位，使其移动到对应字节的高 4 位。另一方面，将 x 整体右移 4 位后再与 0x0F0F0F0F 相与，可以提取原来每个字节的高 4 位并将其移动到低 4 位。最后将这两部分进行按位或运算，即可完成每个字节内部两个 4 位半字节的交换。

## P7 secondLowestZeroBit

目标是找出从低位开始数第二个 0 所在的位置，并只保留该位置的一位 1。首先对 x 按位取反得到  $\sim x$ ，使原数中的 0 全部变成 1，这样问题就转化为寻找  $y = \sim x$  中第二低的 1。利用  $y \& (y - 1)$  可以清除 y 中最低的 1，其中  $y - 1$  用  $y + \sim 0$  实现，得到  $t = y \& (y + \sim 0)$ ，此时原先最低的 1 被删除，其它的 1 仍然被保留。随后利用  $t \& (-t)$  提取 t 中最低的 1，并用  $t = \sim t + 1$  实现相反数。这样最终得到的单个 1 所在位置，就是原数中第二低的 0 所在位置。

## P8 oddParity

目标是判断 x 的 32 位二进制表示中 1 的个数是奇数还是偶数；题目规定当 1 的个数为偶数时返回 1，奇数时返回 0。利用异或具有“相同为 0、不同为 1”的性质，可以不断把多个比特的奇偶性合并。首先将 x 与  $x \gg 16$  异或，高 16 位和低 16 位每一位若都为 1 或都为 0 则得到 0，说明有偶数个 1；不同则为 1，说明有奇数个 1。这一步骤使得数字位数减少了一半，但是 1 的个数的奇偶性没有发生变化。再依次与右移 8 位、4 位、2 位和 1 位的结果异或，最终所有 32 位中 1 的奇偶性都会被压缩到最低位。若最低位为 0，说明原数中 1 的个

数为偶数；若最低位为 1，说明为奇数。最后对最低位取逻辑非，使偶数个 1 返回 1、奇数个 1 返回 0。

## P9 rotateRightBits

目标是将 32 位整数循环右移  $n$  位，即从最低位移出的比特重新补到最高位。首先利用  $n \& 31$  得到实际旋转位数，使任意非负  $n$  都等价于对 32 取模。右半部分通过  $x \gg \text{shift}$  得到，但由于这是算术右移，因此还需要构造高  $\text{shift}$  位为 0 的掩码，把符号扩展产生的 1 清除，从而得到逻辑右移部分。另一部分需要把原数低  $\text{shift}$  位移动到最高端，因此计算  $32 - \text{shift}$  对应的左移位数，并用  $x \ll \text{leftShift}$  完成。最后将逻辑右移部分和左移补回部分按位或，即可实现循环右移。

## P10 roundEvenPow2

目标是把非负整数  $x$  舍入到最接近的  $2^n$  的整数倍，并在恰好位于两个整数倍中间时舍入到偶数。首先计算  $q = x \gg n$ ，它表示  $x$  除以  $2^n$  后的整数部分，低  $n$  位则表示余数。中点大小为  $2^{(n-1)}$ ，即  $\text{half} = 1 \ll (n-1)$ 。普通四舍五入可以通过在  $x$  上加  $\text{half} - 1$  后再右移  $n$  位实现，而为了处理恰好位于中点的情况，还需要根据  $q$  的最低位决定是否额外加 1：若  $q$  为奇数，则中点时应向上舍入，使新的商变为偶数；若  $q$  为偶数，则中点时保持不变。因此偏置量设为  $\text{half} - 1 + (q \& 1)$ ，最后加上偏置后右移  $n$  位并再左移  $n$  位，即可清除低  $n$  位并得到满足规则的  $2^n$  整数倍。

## P11 midpointTowardFirst

目标是在不发生整数溢出的情况下求  $x$  和  $y$  的中点，并且当中点恰好落在两个整数之间时，选择更靠近第一个参数  $x$  的那个整数。首先不能直接计算  $(x+y)/2$ ，因为当  $x$  和  $y$  都很大或都很小时， $x+y$  可能超出 32 位有符号整数范围。因此利用了  $x+y = 2(x \& y) + (x \wedge y)$  这一关系， $x \& y$  保存两数相同为 1 的部分， $x \wedge y$  保存两数不同的部分，避免了直接做  $x+y$ 。接着通过  $(x \wedge y) \& 1$  判断  $x$  和  $y$  的奇偶性是否不同：如果结果为 1，说明一个是奇数、一个是偶数，因此真实中点带 0.5；如果结果为 0，则中点本身就是整数，不需要额外调整。最后还要判断  $x$  和  $y$  的大小，因为基础中点是向下取整得到的：当  $x < y$  时，向下取整的结果本来就更接近  $x$ ；当  $x > y$  且中点带 0.5 时，则需要在基础结果上加 1，才能选择更靠近  $x$  的那个整数。为了避免直接用  $y-x$  比较异号整数时发生溢出，先通过  $x \gg 31$  和  $y \gg 31$  取得符号信息：若两数异号，则直接根据符号判断大小；若两数同号，再利用  $y-x$  的符号判断  $x$  是否大于  $y$ 。最终只在“ $x > y$ ”与“中点带 0.5”这两个条件同时满足时加 1，从而实现题目要求。

## P12 isBetweenEitherOrder

目标是判断  $x$  是否位于  $a$  和  $b$  构成的闭区间内，并且不要求  $a$  和  $b$  按从小到大的顺序给出。因此需要同时判断  $a \leq x \leq b$  和  $b \leq x \leq a$ ，只要其中一种成立就返回 1。为了避免直接通过减法比较不同符号整数时发生溢出，先提取  $x$ 、 $a$ 、 $b$  的符号位；若两个数符号不同，则可以直接根据符号判断大小，负数一定小于非负数；若符号相同，再通过两数之差的符号位判断大小。分别得到  $a \leq x$ 、 $x \leq b$ 、 $b \leq x$  和  $x \leq a$  四个条件后，将前两个相与得到  $a \leq x \leq b$ ，将后两个相与得到  $b \leq x \leq a$ ，最后将两种情况按位或，即可判断  $x$  是否位于两个端点之间。

## P13 mul5Sat

目标是计算  $x \times 5$ ，并在结果发生有符号整数溢出时返回对应的 `INT_MAX` 或 `INT_MIN`。首先利用  $x \ll 2$  计算  $4x$ ，再与  $x$  相加得到  $5x$ 。由于左移和加法都可能发生溢出，因此分别进行检测：对于  $x \ll 2$ ，如果再右移 2 位后不能恢复原来的  $x$ ，说明高位在左移过程中被丢失，发生了溢出；对于  $4x+x$ ，如果两个加数符号相同但结果符号与它们不同，则说明加法溢出。只要其中任意一种溢出条件成立，就根据  $x$  的符号选择饱和值：正数溢出返回 `0x7FFFFFFF`，负数溢出返回 `0x80000000`。最后利用全 0 或全 1 掩码在正常乘积和饱和值之间进行选择，而不使用 if 语句。

## P14 classifyAdd3

目标是在不能使用更宽整数类型的情况下判断精确数学和  $x+y+z$  是否超出 32 位有符号整数范围：大于 `INT_MAX` 返回 1，小于 `INT_MIN` 返回 -1，否则返回 0。实现时将每个整数拆成最高符号位和低 31 位。三个低 31 位作为非负数相加，并记录相加过程中产生的进位数量  $q$ ；另一方面，三个原数中每出现一个负数，其最高位就相当于对整体数值贡献一个  $-2^{31}$ ，因此统计负数个数  $k$ 。这样，精确和相对于正常 32 位范围的高位偏移可以通过  $q-k$  判断：若该差值大于 0，说明产生了额外的正向高位，精确和超过 `INT_MAX`；若该差值小于 -1，说明负向高位过多，精确和低于 `INT_MIN`；只有差值为 0 或 -1 时，结果仍能够由 32 位有符号整数表示。最终根据这个差值分别返回 1、-1 或 0。

## P15 floatScaleThreeHalves

目标是对单精度浮点数  $f$  计算  $f \times 3/2$ ，并直接返回结果的 32 位二进制编码，同时采用 round-to-nearest-even 舍入方式。首先从 `uf` 中分离符号位、阶码和尾数。如果阶码为 `0xFF`，说明输入为 NaN 或无穷大，其中 NaN 要求原样返回，因此直接返回 `uf`。对于非规格化数，

其有效数没有隐藏的最高位 1，可以直接将尾数乘 3 后除以 2，并根据被除掉的最低位判断是否需要进行偶数舍入。对于规格化数，先补上隐藏的最高位 1 得到完整有效数，再乘 3。乘 3 后如果有效数仍处于当前规格化范围，则直接除以 2；如果超过范围，需要多右移一位并将阶码加 1。每次右移都会检查被舍弃部分与中点的关系，在恰好等于中点时根据保留结果最低位的奇偶性决定是否进位。若阶码增加后达到 0xFF，则结果溢出为对应符号的无穷大。

## P16 floatRoundEven

目标是将 `uf` 表示的单精度浮点数舍入到最接近的整数，并仍以单精度浮点數位表示返回；当小数部分恰好为 0.5 时采用舍入到偶数，同时保留正零和负零的符号。首先提取符号位、阶码和尾数。若阶码为 0xFF，说明输入为 NaN 或无穷大，直接原样返回；若阶码足够大，使该浮点数本身已经没有小数位，也直接返回原值。对于绝对值小于 0.5 的数，舍入结果为带原符号的 0；对于绝对值位于 0.5 到 1 之间的数，根据是否恰好等于 0.5 决定返回 0 还是 1，并在中点时遵循偶数规则。对于其余仍含小数部分的数，根据阶码计算需要舍弃的尾数位数，用掩码分离“保留部分”和“被舍弃部分”，再比较被舍弃部分与一半的位置：大于一半时进位，小于一半时不进位，恰好等于一半时只有最低保留位为 1 才进位，从而保证结果为偶数。

## P17 float\_i2f

目标是将 32 位有符号整数 `x` 转换为单精度浮点数的 32 位二进制编码。首先单独记录符号位，并取得 `x` 的绝对值；对于 `x=0` 直接返回 0。随后从最高位开始寻找绝对值中最高的 1，其位置 `p` 决定浮点数的指数，因此阶码为 `p+127`。若整数有效位数不超过 24 位，则所有有效位都可以被单精度浮点数精确保存，只需将整数适当左移，使最高有效位对齐到浮点数隐藏的最高位，并取出低 23 位作为尾数。若有效位数超过 24 位，则必须舍弃一部分低位，因此先右移保留最高 24 位，再记录被舍弃部分。通过比较被舍弃部分与“恰好一半”的值实现 `round-to-nearest-even`：大于一半时进位，小于一半时不进位，恰好一半时根据保留结果最低位是否为 1 决定是否进位。若舍入导致 24 位有效数产生新的最高位，还需要将阶码再加 1 并重新规格化。最后将符号位、阶码和尾数组合成最终 32 位浮点表示。

## P18 bitCount

目标是统计 32 位整数 `x` 的二进制表示中 1 的总个数。采用并行分组计数的方法，而不是逐位循环。首先构造 0x55555555 掩码，将每相邻 2 位分成一组，分别统计每组中 1 的数

量；然后利用 0x33333333 掩码将结果按 4 位分组，把相邻两个 2 位计数相加；接着利用 0x0F0F0F0F 掩码把结果按 8 位分组，得到每个字节中 1 的数量。之后继续将高 8 位计数与低 8 位计数相加，再将高 16 位与低 16 位相加，最终最低若干位中保存整个 32 位整数中 1 的总数。由于 32 位整数最多只有 32 个 1，因此最终只需保留低 6 位即可得到正确结果。

## P19 bitReverse

目标是将 32 位整数的所有二进制位完全逆序，即原来的第 31 位变成第 0 位，第 30 位变成第 1 位，以此类推。实现时采用分治交换的方法，不直接逐位处理。首先构造若干交替掩码，将相邻的 1 位两两交换；然后将相邻的 2 位块交换，再交换 4 位块、8 位块，最后交换高 16 位和低 16 位。每一步都利用掩码分别提取对应的两组位，再通过左移或右移把它们交换位置并重新按位或组合。经过 1 位、2 位、4 位、8 位和 16 位五个层次的交换后，所有 32 个位的位置都被完全反转，从而得到最终的 bitReverse 结果。

## 二、实验总结

本实验把补码、位运算和浮点数 32 位二进制编码中原本较抽象的规则转化成了具体的程序实现。整数部分最重要的技巧是利用符号扩展、掩码和补码恒等式，在不能使用条件语句和减号的情况下完成判断与选择；浮点部分则需要从符号位、阶码和尾数三个字段出发，特别注意非规格化数、NaN、无穷大以及舍入方式等边界情况。

P15~P17 涉及 IEEE754 单精度浮点表示以及 round-to-nearest-even 舍入规则，相关浮点表示与舍入原理参考 Goldberg 对 IEEE 浮点运算的系统总结<sup>[1]</sup>。P19 采用分层交换的方式实现 32 位比特反转，该类 bit-reversal 操作也广泛出现在 FFT 中的位反转排列过程<sup>[2]</sup>。

通过规则检查与 btest 两套机制也可以体会到，代码合法和结果正确是两个独立要求：前者保证实现满足题目限制，后者验证所有输入范围内的行为。实际调试时，先用编译器排除语法与变量问题，再用单题 btest 定位逻辑错误，最后进行全量检查，是比较高效的流程。

## 三、建议

本实验整体设计较为完整，从整数补码、位运算逐步过渡到 IEEE754 浮点表示，题目难度层次比较清晰。建议后续可以在实验说明中增加少量典型位运算技巧的引导，例如掩码构造、利用消去最低位 1、round-to-nearest-even 的判断方式等，让同学在不直接获得答案的情况下更容易找到解题方向。另外，可以对浮点题补充一张 IEEE754 单精度格式示意图，并给出规格化数、非规格化数、NaN 和无穷大的简单分类说明，这样会更方便理解 P15~P17。

#### 四、参考文献

- [1] Goldberg D. What Every Computer Scientist Should Know About Floating-Point Arithmetic[J]. ACM Computing Surveys, 1991, 23(1): 5-48.
- [2] Rutkowska U. A Simple Algorithm for the Bit-Reversal Permutation[J]. Signal Processing, 1991, 23(3): 313-317.